



ALBUKHARY INTERNATIONAL UNIVERSITY

Course Code	CCS2323	Course Name	Software Secured Development
Lecturer	Assoc. Prof. Ts. Dr. Leelavathi Rajamanickam		
Semester/Year	3/2025	Date	05/09/2025
Assessment	Project (20%)	Duration	3 Weeks
Student Id	AIU22102236	Student Name	Md Mehedi Hasan
Student Id	AIU23102301	Student Name	MD PARVEJ AHMED RAFI

Table of Contents

Executive Summary.....	3
1. Introduction.....	3
2.1 STRIDE Threats.....	3
2.2 Asset Classification & CIA Mapping.....	5
2.3 Security Requirements.....	6
2.3.1 Functional security requirements.....	6
2.3.2 Non-Functional security requirements.....	6
3.2 Points of Vulnerability & Mitigations.....	7
3.3 Misuse/Abuse Cases (two critical).....	8
3.3.1 Change Grade Without Authorization.....	8
3.3.2 Falsify Fee Payment Status.....	8
4. Secure Coding Guidelines & Database Security.....	8
4.1 Secure Coding Guidelines.....	8
4.2 Database Security (SQL & Command Injection Prevention).....	9
4.3 Memory Safety (stack/heap overflow prevention).....	9
5. Security Testing & Risk Management.....	10
5.1 Security Testing Plan.....	10
5.2 Black-Box vs White-Box Fuzzing (Portal Context).....	10
5.3 Risk Management Framework (Post-Deployment).....	10
6. Security testing checklist.....	11
7. Conclusion.....	12
8. References.....	13

Executive Summary

The present report suggests a defense-in-depth proposal of a Secure Student Portal to support online course registration, secure fee payment, results access and multi-party communication. It uses threat analysis based on STRIDE, assets-to-CIA triad, functional/non-functional security requirements, design a secure architecture with trust boundaries and controls, define secure coding and database security practices, and provide a high level comprehensive security testing plan and risk management after deployment. (Scope and parts A through D with diagrams/tables per deliverables

1. Introduction

Students portals are now a key element of the academic experience as digital transformation becomes more and more popular in the higher education sector. Such portals are the main means through which students, faculty, and administrators can process course registrations, view results, make payments, and communicate with multiple parties. But with such sensitive data as personal identifiers and academic performance, not mentioning the transactions related to payment, they are an ideal victim of cyber threats. In order to overcome these threats, this report suggests a defense in depth security architecture of a secure student portal. The method is based on structured threat modeling (STRIDE), a strict asset-to-CIA (Confidentiality, Integrity, Availability) classification, and clearly defined security requirements, both functional and non-functional. The suggested architecture focuses on trust boundaries, secure coding and database practices, and layer-based controls to reduce common and sophisticated attack vectors. In addition to architecture, the report describes in detail security testing approaches, such as static and dynamic analysis, fuzzing, penetration testing, and ongoing monitoring. Risk management practices based on international standards (including NIST SP 800-53 and ISO/IEC 27005) can maintain resilience, speedy incident reaction, and adherence to regulatory requirements, even after deployment. By doing so, this report aims to present an integrated roadmap, which does not only protect sensitive academic and financial assets, but also the availability, integrity, and privacy of student and institutional data in a dynamic and threat prone online world.

2. Security Analysis & Requirements

2.1 STRIDE Threats

Scope entities: student, faculty, admin, browser/mobile, Web App, API Gateway, Auth Service/SSO, Registration/Results/Payments services, DB, object storage, external Payment Gateway

STRIDE	Example	Threat	Impact	Primary Mitigations
--------	---------	--------	--------	---------------------

	(Portal Context)		
Spoofting	Credential stuffing on student accounts; session hijacking	Unauthorized access	MFA, passwordless or strong passwords with Argon2id/bcrypt; device binding; short-lived tokens; secure cookies (HttpOnly, Secure, SameSite)
Tampering	Altering course registration payloads or grade records	Integrity loss	End-to-end input validation, server-side object-level authorization (RBAC/ABAC), digital signatures for grade changes, DB row-level security, immutable audit log
Repudiation	Users deny fee payments or grade edits	Dispute risk	Time-stamped, append-only audit logs; request/response signing; payment provider webhooks with nonces; log integrity (hash chains)
Information Disclosure	Transcript/PII leakage via IDOR/XSS, misconfigured CORS	Privacy breach	Strict access checks on object IDs, output encoding + CSP, least-privilege APIs, encryption in transit (TLS 1.3) and at rest (KMS)
Denial of Service	API flood during course add/drop; resource exhaustion	Availability loss	WAF + rate limiting, autoscaling, circuit breakers, cache for reads, per-user quotas; queueing for bursts
Elevation of Privilege	Normal user escalates	Full compromise	Defense-in-depth

	to admin via vulnerable admin endpoints		authorization checks, separate admin plane, strong SSO with step-up auth, deny-by-default firewalling
--	---	--	--

2.2 Asset Classification & CIA Mapping

Asset	C	I	A	Notes
Student PII (name, ID, contact)	High	Med	Med	Privacy laws; breach notification exposure
Academic Records (grades, transcripts)	High	High	Med	Integrity is critical (no unauthorized edits)
Authentication Secrets (password hashes, tokens)	High	High	Med	Hash with Argon2id/bcrypt ; rotate tokens
Payment Data (tokens, receipts)	High	High	Med	Never store PANs; rely on gateway tokens
Course Registration State	Med	High	High	Accuracy & uptime during add/drop
Admin & Faculty Notes	High	High	Med	Sensitive feedback/comme nts
Logs & Audit Trails	Med	High	High	Must be available and tamper-evident

2.3 Security Requirements

2.3.1 Functional security requirements

Authentication: SSO/OIDC including MFA; policy: passwordless or strong password; account lockout and risk-based step-up.

Authorization: Fine-grained RBAC/ABAC; object-level checks on each read/write (e.g. only a student can read his transcript; faculty can update only sanctioned grade records).

Session Management: short-lived access tokens; refresh tokens tied to device; secure cookie flags.

Data Protection: TLS 1.3 in all locations; encryption at rest, supported by KMS; key rotation; field-level encryption of grade/PII hot fields.

Auditability: Grade change, payment and administrative audits, Append only audit trails; tamper-evident hashing.

Payments: refund to attested gateway; signed, idempotent webhook; reconcile ledger every day.

Privacy Controls: User consent disclosures; data minimization; right-to-access/export.

2.3.2 Non-Functional security requirements

Availability: 99.9% when registering a resource; autoscaling and rate limits.

Stability: Graceful failure; circuit breakers; bulkheads.

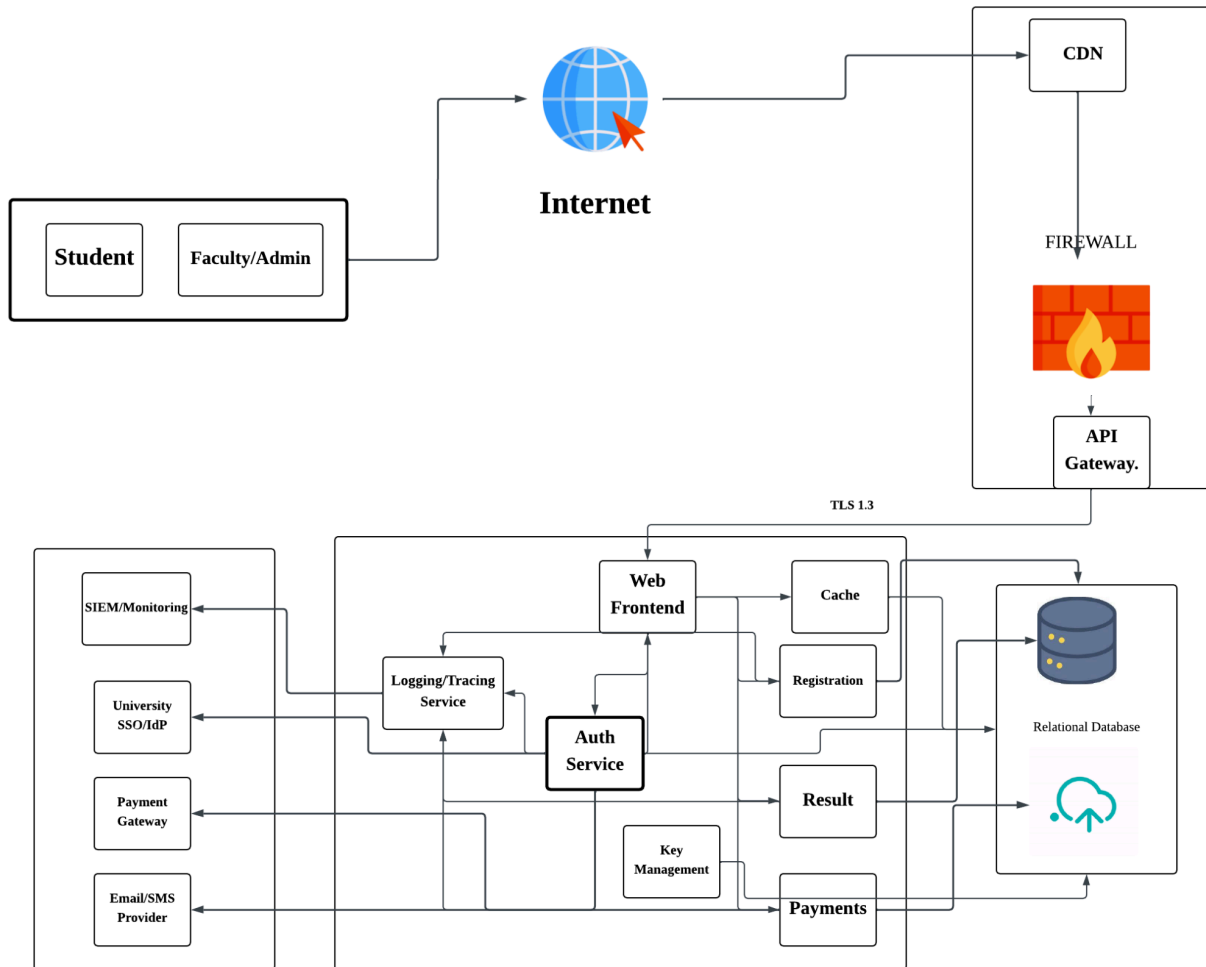
Compliance: Conform to OWASP ASVS, OWASP Top 10, and (where appropriate) PCI DSS (tokenized).

Monitorability: centralized logging, metrics, traces; alerting, and on-call.

Secure SDLC secure scanning: SAST/DAST/SCA/secret scanning; threat modeling at design gates; required code review.

3. Secure Architecture & Design

3.1 Secure Architecture Diagram



3.2 Points of Vulnerability & Mitigations

IDOR on results/registrations: enforce object check on the server side (user-to-record mapping), deny-by-default.

XSS in messages: output encoding, strict CSP, sanitize/allowlist, no HTML in messages.

CSRF on state-changing applications: same-site cookies + anti-CSRF tokens + double-submit pattern.

Weak password reset: time-based, one-time tokens; step-up MFA; anti-enumeration responses.

Webhook signing: check HMAC signatures, hardened source IPs, replay (nonce + timestamp).

SQL injection: parameterized queries/ORM, separation of read and write, least-privileged DB users.

Dependency risks: SCA, with fail-the-build on critical CVEs; pin and verify hashes.

CORS misconfig: tight origin allowlist; no credentials wildcard.

SSRF (file uploads, URL fetch): block metadata IPs, egress allow-listing, content-type and size checks, AV scan.

Add/drop DoS: WAF rules, per-ip/user rate limits, token bucket, queueing, autoscale.

3.3 Misuse/Abuse Cases (two critical

3.3.1 Change Grade Without Authorization

- Abuser goals: Read/modify another student's grade.
- Abuse path: Predictable ID → IDOR → PUT /grades/{studentId}.
- Mitigations: Object-level authorization; signed grade-change requests; dual-control (faculty + head approval); immutable audit log with alerts.

3.3.2 Falsify Fee Payment Status

- Abuser goals: Mark unpaid as paid.
- Abuse path: Forge webhook; replay old success callback.
- Mitigations: HMAC-verified webhooks, strict idempotency keys, transaction reconciliation job, alert on state mismatch.

4. Secure Coding Guidelines & Database Security

4.1 Secure Coding Guidelines

Authentication: OIDC versus university SSO; MFA (TOTP/WebAuthn). Argon2id (or bcrypt) hash passwords, unique per-user salt; lockout and cooldown on failures; device fingerprinting; secure cookies (HttpOnly, Secure, SameSite=Lax/Strict).

Temporary access tokens in the form of JWT; rotating refresh tokens; audience/issuer verification.

Input Validation : Favor allowlists; strict type and length and format checking; server side enforcement. Canonicalization/pre-validation; sanitization of uploads (MIME + magic bytes). Output (HTML, JS, CSS, URL contexts) encoding to avoid XSS.

Error Handling & Logging: Return generic user errors; never leak stack traces or secrets. Structured logs (trace IDs); scrub PII/secrets; rate-limit repetitive logs; secure transport to SIEM; WORM retention for audits.

4.2 Database Security (SQL & Command Injection Prevention)

Database queries should always use parameterized statements or ORM to prevent SQL injection. Roles for read and write must be separated with least-privilege access and row-level isolation. Sensitive data must be encrypted at rest and at the field level, with keys managed via KMS. Access should be restricted through firewalls/permit-lists, with TLS connections and secrets stored in a secure vault. To prevent command injection, avoid shell calls; if unavoidable, use safe APIs with strict allowlists and never pass user input directly.

4.3 Memory Safety (stack/heap overflow prevention)

Use services in memory-safe languages (Java/Kotlin, C#, Go, Rust).

When C/C++ (e.g., agents): bounds-checked APIs, sanitizers (ASan and UBSan), stack canaries, DEP/ASLR, fuzz of critical parsers, zeroizing secrets, no more strcpy/sprintf, use safe alternatives.

5. Security Testing & Risk Management

5.1 Security Testing Plan

Across SDLC

- **Design:** Threat modeling (STRIDE) on both DFDs abuse case review architecture security review.
- **Build:** SAST on all PRs; secret scanning; SCA dependencies scanning IaC scanning cloud baselines.
- **Test (pre-prod):** DAST vs staging; API security test; idor authz test harness fuzzing (see D2); container image scanning misconfig (CIS benchmarks).
- **Pre-release:** Manual penetration test (internal + external scope) business-logic testing (enrollment windows, payment flows).
- **Post-release:** Continuous scanning, chaos exercises, red team exercises on registration week.
- **Penetration Testing Scope:** auth flows, session management, registration/result endpoints, messaging, administration plane, payment webhooks, file upload. Provide: commentary (CVSS), PoCs, reproduction instructions, risk rating, remediation plan.
- **Threat Modeling Artifacts:** DFD including trust boundaries; STRIDE per component; mitigation mapping and register of residual risks.

5.2 Black-Box vs White-Box Fuzzing (Portal Context)

Black-Box Fuzzing: send malformed/random data to HTTP APIs and file upload endpoints and doesn't know anything about the code. Pros: can be easy to get going, mimics attacker; Cons: lower code coverage, can be missing deep logic.

White-Box Fuzzing: instrument services (e.g., request/JSON parsers) to drive mutations through coverage. Pros: better coverage, detects parser/edge-case bugs; Cons: requires more time to set up, and requires access to code/build.

Plan: fuzz API black-box against staging (rate-limited) and white-box fuzz harnesses of JSON/XML parsers and webhook handlers; gate release on zero criticals.

5.3 Risk Management Framework (Post-Deployment)

Framework: ISO 27005/ NIST-like risk register containing a likelihood-impact score; owners and due date.

Vulnerability Mgmt: 30-day high SLA, 7-day critical SLA; emergency patch procedure; SBOM up to date.

Monitoring/Detection: SIEM to centralized logs; auth anomaly, suspicious grade changes, payment mismatch, anomaly-based detection, periodic control validation.

IR & BCDR: Incident playbooks (compromised account, webhook abuse); DB RTO/RPO targets; encrypted and tested backups; tabletop exercises every term.

Change management: Lock change windows outside registration peaks; rollback feature flags.

6. Security testing checklist

Phase	Activity	Details / Scope	Expected Outcome
Design	Threat Modeling	STRIDE analysis on DFDs, abuse case review, architecture security review	Identified threats, mapped mitigations, documented residual risks
Build	Static Application Security Testing (SAST)	Run on all PRs; enforce coding standards	Detect insecure code patterns early
	Secret Scanning	Scan for API keys, credentials in code	Prevent exposure of secrets
	Software Composition Analysis (SCA)	Dependency vulnerability scanning	Ensure no critical CVEs; fail build if found
	IaC / Cloud Baseline Scanning	Check Infrastructure-as-Code for misconfigurations	Ensure secure deployment baselines
Test (Pre-Prod)	Dynamic Application Security Testing (DAST)	Run against staging environments	Detect runtime vulnerabilities
	API Security Testing	Authentication, authorization, IDOR test harness	Prevent logic and access control flaws
	Fuzzing	Black-box (API/file upload) + White-box (parser/webhook handlers)	Detect crashes, memory safety issues
	Container & Config Scanning	Scan for misconfigurations (CIS benchmarks)	Harden container & infrastructure

Pre-Release	Manual Penetration Testing	Internal + external scope; test enrollment windows, payment flows, business logic	Report CVSS score, PoC, risk rating, remediation plan
Post-Release	Continuous Security Scanning	Automated vulnerability scanning, regression monitoring	Detect new issues proactively
	Red Team / Chaos Testing	Attack simulations during peak (e.g., registration week)	Validate resilience and incident response
	Monitoring & Detection	SIEM alerts: suspicious grade changes, payment mismatches, anomalies	Early detection of attacks
	Risk Management	Apply ISO 27005 / NIST risk register; track remediation	Timely patching (7-day SLA for criticals, 30-day SLA for highs)
	Incident Response & BCDR	Test backup restore, tabletop exercises, playbooks	Ensure recovery and continuity

7. Conclusion

A secure student portal should go beyond simple usability, it must be a layered, defense-in-depth solution that is forward-thinking to address emerging threats, at the same time protecting sensitive academic, financial, and personal information. This report provides a clear baseline towards the development of a resilient system through organized threat analysis based on STRIDE and detailed mapping of assets to the CIA triad and defining functional and non-functional security requirements. The architecture and design we propose focus on trust boundaries, secure code, database protections, and mitigations around common misuse and abuse scenarios, making sure that security is integrated at all portal layers. In addition to the design, a full security testing strategy such as fuzzing, penetration and continuous test helps identify the vulnerability and fix it before it is exploited. Lastly, having a risk management framework based on established standards (ISO 27005, NIST SP 800-53, and OWASP ASVS) supports long-term resiliency, structured vulnerability management, monitoring, and incident response. Through the

combination of these, the Secure Student Portal will be able to achieve confidentiality, integrity, and access to its critical assets and provide a trusted and seamless experience to students, faculty, or administrators. This solution ultimately helps prevent cyber threats, in addition to fulfilling compliance, institutional reputation, and user confidence pillars of a sustainable and secure digital learning ecosystem.

8. References

OWASP. (2021). *OWASP Top 10: 2021*.

OWASP. (2023). *Application Security Verification Standard (ASVS) v4.0.3*.

NIST. (2020). *SP 800-63B: Digital Identity Guidelines*.

NIST. (2020). *SP 800-53 Rev. 5: Security and Privacy Controls*.

ISO/IEC. (2018). *ISO/IEC 27005: Information Security Risk Management*.